

République du Sénégal

Ministère de l'Économie, du Plan et de la
Coopération

Agence Nationale de la Statistique et de la
Démographie (ANSD)

École Nationale de la Statistique et de
l'Analyse Économique Pierre NDIAYE



Pipeline de Traitement des Données de l'Enquête sur la Gestion des Ordures Ménagères

.....

*Documentation Technique Complète : Traitement, Analyse et Automatisation
des Données
Région de Dakar, 2018*

Présenté par :

- Ndeye Khary SALL
- Fatou Soumaya WADE
- Josée Clémence JEAZE NGUEMEZI
- David Christ MEKONTCOU NZONDE

Sous la supervision de :

M. Mamadou MBODJ
Ingénieur Statisticien Économiste

*Élèves Analystes Statisticiens, option
Data Science, 3^e année*

Groupe 5

Année académique : 2025–2026

Table des matières

Avertissement	6
Remerciements	7
Sigles et acronymes	8

I PRÉSENTATION ET ARCHITECTURE DU PIPELINE

9

1 Présentation du projet	10
I Objectifs	10
II Résultats du traitement	11
2 Architecture du pipeline	12
I Séquence d'exécution	12
II Diagramme de flux	12
III Principe de scalabilité	13
3 Structure des fichiers	14

II DOCUMENTATION TECHNIQUE DES MODULES 16

4 Module config.py : configuration centrale	17
I Chemins	17
II Module I : ménage et chef de ménage	17
III Mappings du module II : déchets	18
IV Proxy montant mensuel FCFA	19
V Seuils de contrôle qualité	20
5 Module loader.py : chargement et contrôle qualité	21
I Fonctions principales	21
II Contrôles de cohérence	22

III	Code source	23
6	Module builder.py : construction des variables	26
I	Helpers internes	26
II	Bloc A : ménage (12 variables)	27
III	Bloc B : chef de ménage (19 variables)	28
IV	Bloc C : déchets ménagers (87 variables)	29
IV.1	Sources et natures (II.1–II.2)	30
IV.2	Stockage (II.3–II.6)	30
IV.3	Tri et revente (II.7–II.10)	30
IV.4	Traitement (II.11–II.12)	30
IV.5	Bennes tasseuses : service public (II.13–II.20)	31
IV.6	Service alternatif : entités privées (II.21–II.23)	31
IV.7	Paiement (II.23–II.25)	31
IV.8	Infrastructure quartier (III.1–III.33)	31
V	Bloc D : conséquences sanitaires (14 variables)	32
7	Modules exporter.py et qaqc.py	34
I	exporter.py	34
II	qaqc.py : rapport Excel 6 onglets	35
8	Orchestrateur run_pipeline.py	36
9	API FastAPI, main.py	38
I	Routes	38
II	Flux POST /api/process	38
III	Code source	39
III	VARIABLES, UTILISATION ET DÉPLOIEMENT	42
10	Variables produites et absentes	43
I	Récapitulatif	43
II	Variables absentes	43
11	Guide d'utilisation et déploiement	45
I	Installation	45
II	Option A : ligne de commande	45
III	Option B : plateforme web locale	46
IV	Déploiement sur Render	46

V	Dépendances Python	46
---	------------------------------	----

Liste des tableaux

1.1	Indicateurs clés du pipeline	11
2.1	Modules du pipeline	12
4.1	Mappings du Module I	18
4.2	Mappings du Module II	19
4.3	Calcul du proxy montant mensuel	20
4.4	Seuils QC	20
5.1	Fonctions de loader.py	21
5.2	Contrôles de cohérence appliqués	22
6.1	Variables du Bloc A	28
6.2	Variables du Bloc B	29
6.3	Variables du Bloc D	32
7.1	Fonctions de exporter.py	34
7.2	Structure du rapport QAQC	35
9.1	Routes de l'API	38
10.1	Variables analytiques produites	43
10.2	Variables absentes et documentation	44
11.1	requirements.txt	47

Avertissement

« Les points de vue émis dans ce document sont propres à ses auteurs. L'École Nationale de la Statistique et de l'Analyse Économique (ENSAE) Pierre NDIAYE et l'Unité de Coordination de la Gestion des Déchets Solides (UCG) n'entendent donner aucune approbation ni improbation. »

Remerciements

Nous tenons à exprimer notre profonde gratitude à M. Mamadou MBODJ, Ingénieur Statisticien Économiste, pour son encadrement rigoureux, sa disponibilité et ses conseils avisés tout au long de la réalisation de ce pipeline de traitement de données. Nos remerciements vont également à l'ensemble du corps professoral de l'ENSAE Pierre NDIAYE pour la qualité de la formation reçue, ainsi qu'à l'Unité de Coordination de la Gestion des Déchets Solides (UCG) pour le partage des données de l'enquête sur la gestion des ordures ménagères dans la région de Dakar.

Sigles et acronymes

ANSD : Agence Nationale de la Statistique et de la Démographie.

API : Interface de Programmation Applicative (*Application Programming Interface*).

ASGI : *Asynchronous Server Gateway Interface*.

CM : Chef de Ménage.

CSV : *Comma-Separated Values*.

DTA : Format de fichier natif du logiciel Stata.

ENSAE : École Nationale de la Statistique et de l'Analyse Économique Pierre Ndiaye.

FCFA : Franc de la Communauté Financière Africaine.

QAQC : Contrôle et assurance qualité (*Quality Assurance / Quality Control*).

REST : *Representational State Transfer*.

UCG : Unité de Coordination de la Gestion des Déchets Solides.

XLSX : Format de fichier tableur Microsoft Excel.

Première partie

PRÉSENTATION ET ARCHITECTURE DU PIPELINE

Chapitre 1

Présentation du projet

Ce projet implémente un **pipeline de traitement de données reproductible et scalable** pour l'enquête sur la gestion des ordures ménagères dans la région de Dakar, menée conjointement par l'ENSAE et l'UCG en 2018. Il couvre **2 474 ménages** répartis dans les départements de Dakar, Pikine, Rufisque et Guédiawaye.

L'enjeu principal de ce travail n'est pas seulement de produire une table de données exploitable, mais de garantir que n'importe quel membre de l'équipe, y compris une personne n'ayant pas participé au développement initial, puisse relancer l'intégralité du traitement en toute confiance, comprendre chaque transformation appliquée aux données, et faire évoluer le pipeline pour une future édition de l'enquête sans devoir réécrire le code depuis zéro.

I Objectifs

Six objectifs opérationnels ont guidé la conception du pipeline, du chargement des données brutes jusqu'à la mise à disposition du produit final via une interface accessible à tous :

- Extraire les données brutes au format Stata (**.dta**) depuis un dossier **input/**
- Nettoyer et valider les données (valeurs aberrantes, incohérences, données manquantes)
- Construire **132 variables analytiques** organisées en 4 blocs thématiques
- Exporter la table consolidée en CSV et Stata (**.dta**)
- Produire un rapport de contrôle qualité Excel (QAQC) en 6 onglets
- Exposer l'ensemble via une API REST (FastAPI) avec interface web

Ces six objectifs correspondent en réalité aux six étapes successives détaillées dans les chapitres suivants : chaque module du pipeline répond à l'un d'entre eux, ce qui permet de faire évoluer une étape sans avoir à toucher aux autres.

II Résultats du traitement

Le tableau ci-dessous synthétise les indicateurs clés obtenus lors d'une exécution complète du pipeline sur la base brute de l'enquête 2018. Ces chiffres servent de référence pour valider qu'une future exécution, par exemple après une modification du code, produit un résultat cohérent avec celui-ci.

Indicateur	Valeur
Ménages traités	2 474
Variables brutes	139
Variables analytiques	132
Anomalies QC détectées	49
Durée d'exécution	≈ 2 secondes
Fichiers produits	3 (CSV, DTA, XLSX)

TABLE 1.1 – Indicateurs clés du pipeline

La différence entre les 139 variables brutes et les 132 variables analytiques ne signifie pas qu'une partie de l'information a été perdue : certaines variables brutes ont été combinées entre elles (par exemple pour calculer un score synthétique de niveau de vie), tandis que d'autres, non exploitables ou hors périmètre, n'ont pas été retenues dans la table finale.

Le pipeline tourne en **2 secondes** sur l'ensemble des 2 474 ménages, sans erreur, et produit 3 fichiers de sortie prêts à l'emploi. Cette rapidité permet de relancer le traitement autant de fois que nécessaire pendant la phase de développement, sans coût de temps significatif.

Chapitre 2

Architecture du pipeline

L'architecture du pipeline repose sur un principe simple : chaque module Python a une responsabilité unique et bien délimitée. Cette séparation des responsabilités facilite la maintenance, car une modification dans un module (par exemple corriger un mapping de codes) ne risque pas d'introduire un effet de bord dans un autre module (par exemple l'export des fichiers).

I Séquence d'exécution

Le tableau suivant présente les quatre modules exécutés successivement lors d'un traitement complet, dans l'ordre où ils interviennent.

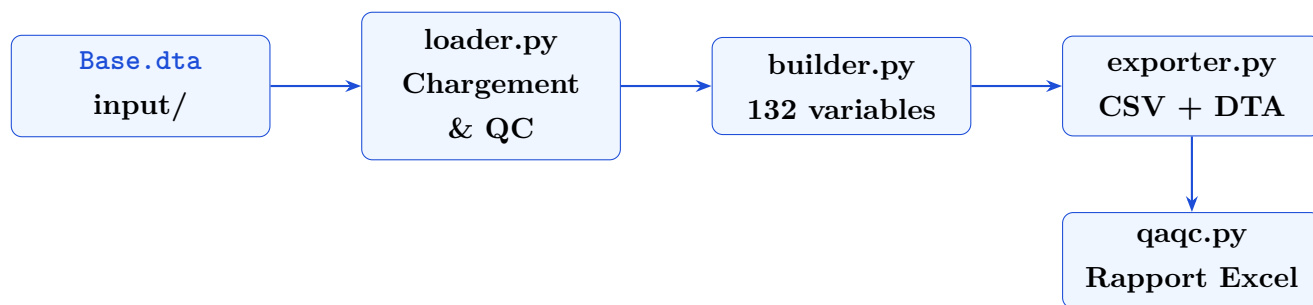
Étape	Module	Rôle
1	<code>loader.py</code>	Chargement du <code>.dta</code> , nettoyage, contrôles qualité
2	<code>builder.py</code>	Construction des 132 variables analytiques
3	<code>exporter.py</code>	Export CSV (UTF-8 BOM) et Stata <code>.dta</code>
4	<code>qaqc.py</code>	Rapport Excel 6 onglets

TABLE 2.1 – Modules du pipeline

Chaque étape dépend strictement de la sortie de l'étape précédente : le module `builder.py` ne peut construire les variables analytiques qu'à partir d'une base déjà nettoyée par `loader.py`, de la même façon que `qaqc.py` ne peut documenter les anomalies qu'une fois la table finale disponible en sortie de `exporter.py`.

II Diagramme de flux

Le schéma ci-dessous illustre visuellement cet enchaînement. La donnée brute entre par la gauche sous forme d'un unique fichier `.dta` et traverse successivement les quatre modules avant d'aboutir aux deux livrables finaux (le jeu de données consolidé et le rapport qualité).



À chaque flèche correspond une transformation précise de la donnée : le passage de `Base.dta` à `loader.py` correspond au nettoyage des types et à la détection des valeurs aberrantes, le passage vers `builder.py` correspond à la construction des variables analytiques, et le passage vers `exporter.py` correspond à la mise en forme finale avant écriture sur disque. Le module `qaqc.py`, placé en aval, consomme à la fois la base brute et la base finale afin de comparer les deux et de documenter les écarts.

III Principe de scalabilité

Toute la logique métier est centralisée dans `config.py`. Ce choix architectural est déterminant : il signifie que la connaissance du questionnaire (les codes de réponse, leurs libellés, les seuils de contrôle qualité) est isolée dans un unique fichier, séparée du code qui manipule réellement les données. Pour une nouvelle édition de l'enquête, la procédure se limite donc à :

1. Mettre à jour `config.py` si les codes ont changé
2. Relancer `python run_pipeline.py`
3. **Aucune modification** des autres modules n'est nécessaire

Cette approche évite le piège classique du code jetable, réécrit à chaque nouvelle édition d'enquête : ici, seul le référentiel de configuration change, tandis que la mécanique de nettoyage, de construction des variables et d'export reste identique.

Chapitre 3

Structure des fichiers

L'organisation des dossiers suit la même logique de séparation des responsabilités que l'architecture logicielle : les données d'entrée, les résultats générés, le code source et l'interface web occupent chacun un emplacement dédié, ce qui évite toute confusion entre ce qui doit être modifié par un développeur et ce qui est produit automatiquement par le pipeline.

```
Groupe_Insalubrite_traitement_donnees/  
|-- input/                <- Donnees brutes (non versionnees)  
|   '-- Base.dta  
|-- output/               <- Resultats generes automatiquement  
|   |-- dataset_dechets.csv  
|   |-- dataset_dechets.dta  
|   |-- QAQC_rapport.xlsx  
|   '-- pipeline_YYYYMMDD.log  
|-- src/                  <- Code source du pipeline  
|   |-- config.py         <- Mappings et parametres centralises  
|   |-- loader.py         <- Chargement et controles qualite  
|   |-- builder.py        <- Construction des variables  
|   |-- qaqc.py           <- Rapport qualite Excel  
|   |-- exporter.py       <- Export CSV et Stata  
|   '-- run_pipeline.py   <- Orchestrateur principal  
|-- templates/  
|   '-- index.html        <- Interface web  
|-- main.py               <- API FastAPI  
|-- requirements.txt  
|-- render.yaml  
'-- README.md
```

Listing 3.1 – Arborescence du projet

Note

Les dossiers `input/` et `output/` sont dans `.gitignore` : les données ne sont pas versionnées sur GitHub.

Ce choix protège la confidentialité des données de l'enquête (qui ne doivent pas circuler sur une plateforme publique) tout en garantissant que le dépôt Git ne contient

que ce qui est réellement nécessaire pour faire fonctionner le pipeline, à savoir le code source lui-même.

Deuxième partie

DOCUMENTATION TECHNIQUE DES MODULES

Chapitre 4

Module `config.py` : configuration centrale

Ce fichier centralise **tous** les mappings de codes, labels et paramètres. C'est le seul fichier à modifier entre deux éditions de l'enquête, et c'est pour cette raison qu'il est présenté en premier dans cette documentation technique : comprendre son contenu permet de comprendre l'ensemble du comportement du pipeline sans avoir à lire le code des autres modules.

I Chemins

Les quatre constantes ci-dessous définissent l'emplacement des dossiers d'entrée et de sortie de façon relative à l'emplacement du fichier `config.py` lui-même. Cette relativité est volontaire : elle permet au pipeline de fonctionner quel que soit l'endroit où le projet est copié sur le disque, sans qu'un chemin absolu codé en dur ne vienne casser l'exécution sur une autre machine.

```
1 BASE_DIR      = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
2 INPUT_DIR     = os.path.join(BASE_DIR, "input")
3 OUTPUT_DIR    = os.path.join(BASE_DIR, "output")
4 INPUT_FILE    = "Base.dta"
```

Notons que l'API FastAPI, décrite au chapitre 9, modifie temporairement `INPUT_DIR` et `OUTPUT_DIR` à l'exécution afin d'isoler chaque requête utilisateur dans son propre répertoire temporaire.

II Module I : ménage et chef de ménage

Les données collectées sur le terrain via ODK ou Stata utilisent des codes numériques (1, 2, 3...) plutôt que des libellés explicites, pour des raisons de rapidité de saisie. Le rôle de ces mappings est de traduire ces codes en modalités lisibles, afin que la table finale et le rapport QAQC soient directement compréhensibles par un utilisateur qui n'a pas participé à la conception du questionnaire.

Variable	Codes et labels
SEXE	1 → Homme 2 → Femme
STATUT_REPONDANT (I.4)	1→CM 2→Femme/Mari 3→Fils/Fille 4→Gendre ... 10→Employé
STATUT_OCCUPATION (I.6)	1→Locataire 2→Propriétaire 3→Logé par un tiers
TYPE_LOGEMENT (I.7)	1→Case/Baraque 2→Maison basse 3→Maison en étage 4→Appartement
TYPE_ENSEIGNEMENT_CM (I.10)	1→Française 2→Arabe 3→Coranique 4→Aucun 5→NSP
NIVEAU_INSTRUCTION_CM (I.11)	1→Primaire 2→Secondaire 3→Supérieur 4→NSP
ALPHA (I.12)	1→Oui 2→Non 3→Ne sait pas

TABLE 4.1 – Mappings du Module I

Si une future édition de l'enquête ajoute une nouvelle modalité de réponse (par exemple un nouveau type de logement), il suffit d'ajouter une entrée dans le dictionnaire correspondant de `config.py` : aucune autre partie du pipeline n'a besoin d'être modifiée.

III Mappings du module II : déchets

De la même façon, le module II du questionnaire (consacré à la gestion des déchets proprement dite) mobilise ses propres codes, résumés ci-dessous. Ces variables couvrent le mode de stockage des ordures au sein du ménage, l'accès au service de collecte public et les modalités de paiement éventuel.

Variable	Codes et labels
MODE_STOCKAGE (II.3)	1→Poubelle 2→Seau/Bassine 3→Fût 4→Carton 5→Sac
PLACE_STOCKAGE (II.5)	1→Cuisine 2→Parcelle 3→Rue proche 4→Rue loin
ACCES_BENNE (II.13)	1→Oui 2→Non
DISTANCE_COLLECTE (II.16)	1→Proche 2→Acceptable 3→Éloignée
FREQUENCE_BENNE (II.19)	1→Régulière 2→Pas régulière
SATISFACTION_BENNE (II.20)	1→Très bon 2→Bon 3→Mauvais 4→Très mauvais
SERVICE_ALTERNATIF (II.22)	1→Charrettes 2→Société pré-collecte 3→Associations
MONTANT_EVACUATION (II.25_1)	1→<200 F 2→200-400 F 3→401-1000 F 4→1001-2000 F 5→>2000 F
FREQUENCE_PAIEMENT (II.25_2)	1→Jour 2→Semaine 3→Mois 4→An

TABLE 4.2 – Mappings du Module II

Deux variables méritent une attention particulière : [ACCES_BENNE](#) conditionne un ensemble de questions suivantes (si le ménage n'a pas accès à la benne, les questions sur la satisfaction du service n'ont pas de sens), et [MONTANT_EVACUATION](#) n'est pas un montant exact mais une tranche, ce qui justifie le calcul d'un proxy détaillé dans la section suivante.

IV Proxy montant mensuel FCFA

Le questionnaire ne demande pas aux ménages un montant exact payé pour le service d'évacuation des déchets, mais une tranche de dépense (inférieure à 200 F, entre 200 et 400 F, et ainsi de suite). Afin de pouvoir malgré tout calculer des statistiques descriptives (moyenne, médiane) sur la dépense mensuelle, le pipeline construit un proxy en combinant la médiane de chaque tranche avec un facteur de conversion qui dépend de la fréquence de paiement déclarée (journalière, hebdomadaire, mensuelle ou annuelle) :

$$\text{montant_mensuel} = \text{médiane}(\text{tranche}) \times \text{facteur}(\text{fréquence})$$

Le tableau ci-dessous détaille ce calcul pour chacune des cinq tranches possibles, avec un exemple concret de montant mensuel obtenu.

Tranche	Médiane (FCFA)	Facteur mensuel	Exemple
1 (< 200 F)	100	$\times 1$	100 F/mois
2 (200–400 F)	300	$\times 4,33$	1 299 F/mois
3 (401–1000 F)	700	$\times 30$	21 000 F/mois
4 (1001–2000 F)	1 500	$\div 12$	125 F/mois
5 (> 2000 F)	2 500	selon fréq.	N/A

TABLE 4.3 – Calcul du proxy montant mensuel

Note

Si `II_24` $\neq 1$ (service non payant), le montant est mis à **0 FCFA**.

Ce proxy reste une approximation statistique et non une mesure exacte de la dépense réelle de chaque ménage : il permet néanmoins des comparaisons pertinentes entre groupes de ménages (par exemple entre quartiers), ce qui correspond à l'usage prévu pour cette variable dans les analyses ultérieures de l'UCG.

V Seuils de contrôle qualité

Les quatre seuils suivants encadrent la plausibilité démographique des réponses (âge et taille du ménage) ainsi que le taux acceptable de données manquantes par variable. Ils ont été fixés à des valeurs larges mais réalistes pour le contexte sénégalais, de façon à détecter les erreurs de saisie manifestes sans pour autant écarter à tort des observations valides mais atypiques.

Paramètre	Valeur	Application
<code>QC_AGE_MIN</code>	15 ans	Âge minimum réaliste CM
<code>QC_AGE_MAX</code>	110 ans	Âge maximum réaliste
<code>QC_TAILLE_MAX</code>	50 personnes	Taille ménage maximale
<code>QC_MISSING_THRESH</code>	50%	Alerte si taux dépasse ce seuil

TABLE 4.4 – Seuils QC

Ces quatre constantes sont directement mobilisées par les fonctions de contrôle qualité du module `loader.py`, présenté au chapitre suivant : elles n'ont donc aucune utilité si elles restent isolées dans `config.py`, leur intérêt réside précisément dans leur réutilisation systématique par le code de nettoyage.

Chapitre 5

Module loader.py : chargement et contrôle qualité

Le module `loader.py` constitue la première étape du pipeline : il est responsable de faire entrer la donnée brute dans le système, de la mettre dans un format exploitable, et de signaler toute anomalie avant que les étapes suivantes ne s'appuient dessus. Une erreur non détectée à ce stade se propagerait silencieusement dans l'ensemble des 132 variables analytiques construites ensuite.

I Fonctions principales

Le tableau ci-dessous présente les six fonctions du module, dans l'ordre logique où elles interviennent lors d'un appel à `run(qc_log)`.

Fonction	Description
<code>load_raw(filepath)</code>	Charge le .dta avec pyreadstat. Journalise dimensions et colonnes.
<code>clean_types(df, qc_log)</code>	Normalise les types : codes string, numériques, binaires.
<code>quality_checks(df, qc_log)</code>	Applique outliers, skip patterns, taux de manquants.
<code>run(qc_log)</code>	Point d'entrée : <code>load_raw</code> → <code>clean_types</code> → <code>quality_checks</code> .
<code>_flag_outlier(...)</code>	Met à NaN les valeurs hors bornes, journalise dans <code>qc_log</code> .
<code>_flag_incoherence(...)</code>	Journalise les incohérences sans modifier les données.

TABLE 5.1 – Fonctions de loader.py

On notera une distinction importante entre `_flag_outlier`, qui corrige activement la donnée en remplaçant une valeur aberrante par NaN, et `_flag_incoherence`, qui

se contente de signaler un problème dans le journal qualité sans toucher à la donnée elle-même. Ce choix est délibéré : certaines incohérences relèvent d'un comportement attendu du questionnaire (voir la section suivante), et il serait risqué de les corriger automatiquement sans risquer de perdre de l'information.

II Contrôles de cohérence

Un questionnaire d'enquête comporte généralement des questions conditionnelles, aussi appelées « skip patterns » : certaines questions ne doivent être posées que si une réponse précédente le justifie. Par exemple, il est inutile de demander la satisfaction vis-à-vis du service de collecte à un ménage qui a déclaré ne pas y avoir accès. Les contrôles suivants vérifient que ces règles de branchement ont bien été respectées lors de la collecte sur le terrain.

Contrôle	Action
Outliers âge répondant (I.3)	Valeurs < 15 ou $> 110 \rightarrow \text{NaN}$
Outliers âge CM (I.9)	Valeurs < 15 ou $> 110 \rightarrow \text{NaN}$
Outliers taille ménage (I.5)	Valeurs < 0 ou $> 50 \rightarrow \text{NaN}$
Skip pattern benne (II.13=2)	Si sans benne, détail II.14–II.20 ne devrait pas être renseigné
Payant sans montant	Si II.24=1 mais II.25_1 manquant $\rightarrow$ alerte
Revente incohérente	Si II.8=2 mais $_v18-_v22 = 1 \rightarrow$ alerte
Dépôts sauvages skip	Si III.12=3 mais III.13/14 renseignés $\rightarrow$ alerte
Taux de manquants	Alerte si une variable dépasse 50% de valeurs manquantes

TABLE 5.2 – Contrôles de cohérence appliqués

Ces huit contrôles ne corrigent jamais la donnée : ils l'enrichissent d'un journal d'anomalies (`qc_log`), qui sera ensuite exploité tel quel par le rapport QAQC présenté au chapitre 7. Ce choix de séparer la détection de la correction laisse à l'analyste final la responsabilité d'interpréter chaque anomalie au cas par cas, plutôt que d'imposer une correction automatique potentiellement inadaptée.

III Code source

Le code source complet du module est reproduit ci-dessous. On y retrouve les deux fonctions utilitaires privées (`_flag_outlier` et `_flag_incoherence`) suivies des trois fonctions principales (`load_raw`, `clean_types`, `quality_checks`), puis de la fonction `run` qui orchestre l'ensemble et constitue le seul point d'entrée appelé depuis l'extérieur du module.

```

1 import os, logging
2 import numpy as np
3 import pandas as pd
4 import pyreadstat
5 from config import (INPUT_DIR, INPUT_FILE, QC_AGE_MIN,
6                     QC_AGE_MAX, QC_TAILLE_MAX, QC_MISSING_THRESH)
7
8 logger = logging.getLogger(__name__)
9
10 def _to_numeric_or_nan(series):
11     return pd.to_numeric(series, errors="coerce")
12
13 def _flag_outlier(df, col, lo, hi, label, qc_log):
14     if col not in df.columns:
15         return df
16     mask = df[col].notna() & ((df[col] < lo) | (df[col] > hi))
17     n = mask.sum()
18     if n > 0:
19         qc_log.append({"check": f"outlier_{col}",
20                      "description": f"{label} : hors [{lo}, {hi}]",
21                      "n_affected": int(n), "action": "Remplace par NaN"})
22         df.loc[mask, col] = np.nan
23     return df
24
25 def _flag_incoherence(df, mask, label, action, qc_log):
26     n = mask.sum()
27     if n > 0:
28         qc_log.append({"check": "incoherence",
29                      "description": label, "n_affected": int(n),
30                      "action": action})
31     return n
32
33 def load_raw(filepath):
34     logger.info(f"Chargement : {filepath}")
35     df, meta = pyreadstat.read_dta(filepath)
36     logger.info(f"Dimensions : {df.shape[0]} x {df.shape[1]}")
37     return df, meta

```

```

38
39 def clean_types(df, qc_log):
40     df = df.copy()
41     str_cat = ["I_4", "I_7", "I_10", "II_3", "II_5", "II_22",
42               "II_23", "III_4", "III_23", "III_26", "III_30", "Commune"]
43     for col in str_cat:
44         if col in df.columns:
45             df[col] = df[col].astype(str).str.strip().replace(
46                 {"": np.nan, "nan": np.nan})
47     num_cols = ["I_3", "I_9", "I_5_1", "I_5_2", "I_5_3", "I_5_4"]
48     for col in num_cols:
49         if col in df.columns:
50             df[col] = _to_numeric_or_nan(df[col])
51     likert = ["I_2", "I_6", "I_8", "I_11", "II_4", "II_6", "II_7",
52             "II_8", "II_13", "II_14", "II_15", "II_16", "II_17",
53             "II_19", "II_20", "II_24", "II_25_1", "II_25_2"]
54     for col in likert:
55         if col in df.columns:
56             df[col] = _to_numeric_or_nan(df[col])
57     for col in [c for c in df.columns if c.startswith("_v")]:
58         df[col] = _to_numeric_or_nan(df[col])
59     logger.info("Nettoyage des types termine.")
60     return df
61
62 def quality_checks(df, qc_log):
63     df = df.copy()
64     df = _flag_outlier(df, "I_3", QC_AGE_MIN, QC_AGE_MAX, "Age repondant",
65                       qc_log)
66     df = _flag_outlier(df, "I_9", QC_AGE_MIN, QC_AGE_MAX, "Age CM", qc_log)
67     df = _flag_outlier(df, "I_5", 0, QC_TAILLE_MAX, "Taille menage", qc_log)
68
69     if "II_13" in df.columns:
70         no_benne = df["II_13"] == 2
71         for col in ["II_14", "II_16", "II_17", "II_18", "II_19", "II_20"]:
72             if col in df.columns:
73                 _flag_incoherence(df, no_benne & df[col].notna(),
74                                   f"Sans benne mais {col} renseigne",
75                                   "Conserve -- skip pattern", qc_log)
76     if "II_24" in df.columns and "II_25_1" in df.columns:
77         _flag_incoherence(df, (df["II_24"] == 1) & df["II_25_1"].isna(),
78                           "Payant mais montant manquant", "NaN conserve", qc_log)
79     for col in df.columns:
80         r = df[col].isna().mean()
81         if r > QC_MISSING_THRESH:
82             qc_log.append({"check": "missing_rate",

```



```
81         "description": f"'{col}' : {r:.1%} manquants",
82         "n_affected": int(df[col].isna().sum()),
83         "action": "Alerte -- verifier skip pattern"})
84     logger.info(f"Controles : {len(qc_log)} signalements.")
85     return df
86
87 def run(qc_log):
88     filepath = os.path.join(INPUT_DIR, INPUT_FILE)
89     df, meta = load_raw(filepath)
90     df = clean_types(df, qc_log)
91     df = quality_checks(df, qc_log)
92     return df, meta
```

Listing 5.1 – loader.py

La liste `qc_log`, passée en paramètre à chaque fonction, est un exemple d'accumulateur partagé : elle est initialisée vide par l'orchestrateur (`run_pipeline.py`) puis enrichie progressivement par chaque contrôle, avant d'être transmise telle quelle au module `qaqc.py` pour générer le rapport final.

Chapitre 6

Module builder.py : construction des variables

Une fois la base nettoyée par `loader.py`, le module `builder.py` prend le relais pour construire les 132 variables analytiques réparties en quatre blocs thématiques. C'est le module le plus volumineux du pipeline, dans la mesure où il concentre toute la logique de transformation métier propre au questionnaire déchets.

I Helpers internes

Avant de détailler les quatre blocs de variables, il convient de présenter les fonctions utilitaires réutilisées d'un bloc à l'autre. Elles répondent toutes au même souci de robustesse : une colonne absente de la base brute (par exemple parce qu'elle n'a pas été collectée cette année-là) ne doit jamais faire planter le pipeline, mais simplement produire une colonne de valeurs manquantes accompagnée d'un avertissement dans les journaux.

```
1 def _col(df, col, default=None):
2     """Acces securise : NaN + warning si colonne absente."""
3     if col in df.columns:
4         return df[col]
5     logger.warning(f"Colonne absente : '{col}'")
6     return pd.Series(default if default is not None
7                       else np.nan, index=df.index)
8
9 def _map(series, mapping):
10     return series.map(mapping)
11
12 def _oui_non(series):
13     return series.map({1: "Oui", 2: "Non"})
14
15 def _binary_flag(series):
16     return series.fillna(0).astype(int)
17
18 def _concat_flags(df, cols_labels, sep=" | "):
19     """Liste textuelle des modalites selectionnees."""
```

```

20     avail = [(c,l) for c,l in cols_labels if c in df.columns]
21     if not avail:
22         return pd.Series(np.nan, index=df.index)
23     def row_labels(row):
24         lbls = [l for c,l in avail if row.get(c,0) == 1]
25         return sep.join(lbls) if lbls else np.nan
26     return df[[c for c,_ in avail]].apply(row_labels, axis=1)
27
28 def _sum_flags(df, cols):
29     return sum(_col(df,c).fillna(0) for c in cols).astype(int)

```

Listing 6.1 – Helpers builder.py

La fonction `_concat_flags` mérite une attention particulière : elle transforme une série de colonnes binaires (une par modalité possible, par exemple une colonne par type de nuisible observé) en une seule colonne textuelle listant les modalités effectivement cochées. C'est ce mécanisme qui produit les variables de type `..._liste` rencontrées dans les blocs suivants.

II Bloc A : ménage (12 variables)

Le bloc A rassemble les caractéristiques structurelles du ménage : sa localisation géographique, la composition de son foyer par âge et par sexe, ainsi que son statut d'occupation et son type d'habitat.

Variable	Source	Description
<code>commune_code</code>	Commune	Code commune
<code>quartier</code>	quartier	Quartier de l'enquête
<code>zone</code>	N/A	Constante : Région de Dakar (urbain)
<code>nb_enfants_0_4</code>	I_5_1	Enfants < 5 ans
<code>nb_enfants_5_15</code>	I_5_2	Enfants 5–15 ans
<code>nb_adultes_H</code>	I_5_3	Hommes adultes (15+)
<code>nb_adultes_F</code>	I_5_4	Femmes adultes (15+)
<code>taille_menage</code>	I_5	Taille totale
<code>statut_occupation</code>	I_6	Locataire / Propriétaire / Logé
<code>type_habitat</code>	I_7	Case / Maison / Appartement
<code>proxy_niveau_vie</code>	I_6, I_7, II_24	Score 0–3
<code>proxy_niveau_vie_label</code>	score	Faible / Moyen-bas / Moyen-haut / Élevé

TABLE 6.1 – Variables du Bloc A

La dernière variable de ce bloc, `proxy_niveau_vie`, mérite une explication : en l'absence de question directe sur le revenu du ménage, un score composite a été construit à partir de trois indicateurs indirects (être propriétaire de son logement, occuper un habitat plus qualitatif qu'une case ou une baraque, et payer le service d'évacuation des déchets), selon la formule suivante :

$$\text{score} = \mathbf{1}[I_6 = 2] + \mathbf{1}[I_7 \in \{2, 3, 4\}] + \mathbf{1}[II_24 = 1]$$

Un score de 0 correspond ainsi à un ménage ne remplissant aucun de ces trois critères (niveau de vie proxy faible), tandis qu'un score de 3 correspond à un ménage les remplissant tous (niveau de vie proxy élevé). Ce score reste, comme son nom l'indique, un proxy : il n'équivaut pas à une mesure directe du revenu, mais fournit un indicateur ordinal utile pour des comparaisons entre ménages.

III Bloc B : chef de ménage (19 variables)

Le bloc B décrit le profil socio-démographique du chef de ménage (CM) ainsi que celui du répondant à l'enquête, qui ne sont pas nécessairement la même personne. Il couvre le sexe, l'âge, le niveau d'instruction et l'alphabétisation, déclinée par langue.

Variable	Source	Description
<code>repondant_sexe</code>	I_2	Sexe du répondant
<code>repondant_age</code>	I_3	Âge (98/99 → NaN)
<code>repondant_statut</code>	I_4	Lien avec le CM
<code>repondant_est_cm</code>	I_4	Répondant = CM ?
<code>cm_sexe</code>	I_8	Sexe du CM
<code>cm_age</code>	I_9	Âge CM (98/99 → NaN)
<code>cm_tranche_age</code>	I_9	<25 / 25-34 / 35-44 / 45-59 / 60+
<code>cm_type_enseignement</code>	I_10	Française / Arabe / Coranique / Aucun
<code>cm_niveau_instruction</code>	I_11	Primaire / Secondaire / Supérieur
<code>cm_alpha_francais</code>	I_12_2	Alphabétisé en français
<code>cm_alpha_anglais</code>	I_12_3	Alphabétisé en anglais
<code>cm_alpha_arabe</code>	I_12_4	Alphabétisé en arabe
<code>cm_alpha_lng_nat</code>	I_12_5	Alphabétisé en langue nationale
<code>cm_est_alphabetise</code>	I_12_2-5	Au moins une langue
<code>cm_situation_matr.</code>	N/A	Valeur manquante (variable non collectée)
<code>cm_statut_emploi</code>	N/A	Valeur manquante (variable non collectée)
<code>cm_secteur_emploi</code>	N/A	Valeur manquante (variable non collectée)
<code>cm_revenu</code>	N/A	Valeur manquante (variable non collectée)

TABLE 6.2 – Variables du Bloc B

Les quatre dernières lignes du tableau appellent une précision importante : ces variables (situation matrimoniale, statut d'emploi, secteur d'emploi, revenu du CM) ne figurent pas dans le questionnaire 2018. Elles sont néanmoins déclarées dans la table finale, remplies de valeurs manquantes, afin de préserver un schéma de colonnes stable d'une édition à l'autre de l'enquête : si une future version du questionnaire ajoute ces questions, il suffira de renseigner leur source dans `builder.py` sans avoir à modifier la structure de la table.

IV Bloc C : déchets ménagers (87 variables)

Le bloc C constitue le cœur thématique de l'enquête : il couvre l'intégralité du parcours des déchets au sein du ménage, depuis leur production jusqu'à leur évacuation, en passant par le stockage, le tri et le paiement éventuel d'un service. Ses 87 variables sont organisées en huit sous-thèmes, détaillés ci-dessous.

IV.1 Sources et natures (II.1–II.2)

Cette première sous-partie identifie d'où proviennent les déchets du ménage (cuisine, élevage, activité commerciale...) et de quelle nature ils sont (organiques, plastiques, verre...). Chaque source ou nature possible est encodée par un indicateur binaire, puis regroupée en une liste textuelle pour faciliter la lecture.

- `src__v1` à `src__v7` : indicateurs binaires 0/1 par source
- `sources_liste` : liste textuelle (ex. « Cuisine | Élevage »)
- `nb_sources` : nombre total de sources
- `nat__v8` à `nat__v17` : indicateurs par nature
- `natures_liste`, `nb_natures`

IV.2 Stockage (II.3–II.6)

Ces variables décrivent la façon dont le ménage conserve ses déchets avant leur évacuation : le type de récipient utilisé, sa localisation dans ou hors de la parcelle, et si sa capacité est jugée suffisante.

- `mode_stockage` : Poubelle / Seau / Fût / Carton / Sac
- `stockage_couvert`, `place_stockage`, `capacite_stockage_suffisante`

IV.3 Tri et revente (II.7–II.10)

Cette sous-partie mesure les pratiques de valorisation informelle des déchets par les ménages eux-mêmes, avant même leur prise en charge par un service de collecte.

- `tri_avant_evacuation`, `revente_dechets`, `types_revendus_liste`, `raison_non_revente`

IV.4 Traitement (II.11–II.12)

Ici, le questionnaire s'intéresse aux pratiques de traitement des déchets directement effectuées par le ménage, en dehors de tout circuit institutionnel : enfouissement dans la parcelle, incinération, recyclage ou compostage domestique.

- `proportion_traitee` : totalité / partie / aucune
- `traitement_enfouissement`, `traitement_incineration`, `traitement_recyclage`, `traitement_compostage`
- `au_moins_un_traitement` : indicateur synthétique (Oui/Non)

IV.5 Bennes tasseuses : service public (II.13–II.20)

Cette sous-partie évalue l'accès au service public de collecte par bennes tasseuses, ainsi que la satisfaction des ménages vis-à-vis de ce service : fréquence de passage, propreté du point de collecte, respect des horaires annoncés.

- `acces_benne_tasseuse`, `benne_usage_regulier`
- `distance_point_collecte`, `benne_point_nettoyage`, `benne_heure_convient`
- `frequence_benne`, `satisfaction_benne`

IV.6 Service alternatif : entités privées (II.21–II.23)

Lorsque le service public n'est pas disponible ou jugé insatisfaisant, les ménages recourent parfois à des solutions alternatives (charretiers indépendants, sociétés de pré-collecte, associations de quartier). Ces variables identifient ce recours et sa motivation.

- `service_alternatif_type` : Charrettes / Pré-collecte / Associations
- `service_alternatif_raison`, `a_service_alternatif`

IV.7 Paiement (II.23–II.25)

Cette sous-partie mesure la disposition à payer et le montant effectivement payé pour le service d'évacuation, à l'origine du proxy de dépense mensuelle présenté au chapitre 4.

- `service_evacuation_payant` : consentement à payer (Oui/Non)
- `montant_evacuation_tranche`, `frequence_paiement`
- `montant_mensuel_evacuation_fcfa` : proxy (0 si gratuit)

IV.8 Infrastructure quartier (III.1–III.33)

Enfin, cette dernière et plus large sous-partie sort du strict cadre du ménage pour documenter l'environnement collectif du quartier : présence de bacs et de corbeilles publiques, dépôts sauvages observés, nuisibles associés à l'insalubrité, et connaissance de l'action de l'UCG par la population.

- `acces_depotoire_normalise` : bac ou corbeille présent (synthétique)
- `existence_bacs_quartier`, `satisfaction_bacs`, `corbeilles_rue_presentes`
- `dep_sauvages_rue_freq`, `depots_sauvages_observes`

- nuisible__v23 à nuisible__v29 : moustiques, mouches, cafards, souris, vers, rats
- nuisibles_liste, nb_nuisibles
- service_balayage_rues, set_setal
- place_publicque, questionnaire_place_publicque, salubrite_place_publicque
- marche_present, questionnaire_marche, gestion_dechets_marche
- connait_UCG, campagne_sensibilisation, satisfait_campagne
- suggestions_amelioration (texte libre)

V Bloc D : conséquences sanitaires (14 variables)

Le dernier bloc établit un lien entre l'insalubrité liée aux déchets et la santé perçue des membres du ménage, à travers une série de maladies et symptômes déclarés au cours de la période récente.

Variable	Source	Description
maladie__v30	__v30	Fièvre (0/1)
maladie__v31	__v31	Asthme (0/1)
maladie__v32	__v32	Rhume (0/1)
maladie__v33	__v33	Sinusite (0/1)
maladie__v34	__v34	Toux (0/1)
maladie__v35	__v35	Nausées/Vomissements (0/1)
maladie__v36	__v36	Démangeaisons (0/1)
maladie__v37	__v37	Aucune maladie (0/1)
maladie__v38	__v38	Autre maladie (0/1)
maladies_liste	__v30–38	Liste textuelle
nb_maladies	__v30–36,38	Nombre de types (hors aucune)
aucune_maladie	__v37	Indicateur 0/1
nb_types_maladies	IV_count	Nombre total dans le ménage
maladie_liee_dechets	__v31,33–36	Oui si asthme/sinusite/toux/nausées/démangeaisons

TABLE 6.3 – Variables du Bloc D

Note

L'indicateur `maladie_liee_dechets` est :

$$\mathbf{1} \left[\sum_{j \in \{31, 33, 34, 35, 36\}} -v_j \geq 1 \right]$$

La fièvre et le rhume sont exclus car non spécifiques aux déchets.

Ce choix méthodologique d'exclure la fièvre et le rhume repose sur une justification épidémiologique simple : ces deux symptômes sont extrêmement courants dans la population générale et ne permettent pas, à eux seuls, d'établir un lien crédible avec l'exposition aux déchets ménagers, contrairement à l'asthme, la sinusite, la toux, les nausées ou les démangeaisons, plus spécifiquement associés à un environnement insalubre.

Chapitre 7

Modules `exporter.py` et `qaqc.py`

Une fois la table analytique construite par `builder.py`, deux modules se partagent la responsabilité de la mise à disposition finale des résultats : `exporter.py` écrit les données proprement dites, tandis que `qaqc.py` documente la qualité de ce traitement dans un rapport dédié.

I `exporter.py`

Le pipeline produit délibérément deux formats de fichiers de données, et non un seul : le format CSV assure une interopérabilité maximale avec Excel, R ou Python, tandis que le format Stata (`.dta`) répond directement aux habitudes de travail des statisticiens de l'ENSAE et de l'UCG, qui utilisent Stata comme outil d'analyse principal.

Fonction	Description
<code>_prepare_for_stata(df)</code>	Raccourcit les noms > 32 caractères (limite Stata).
<code>export_csv(df, filename)</code>	Export CSV UTF-8 avec BOM (<code>utf-8-sig</code>) pour Excel.
<code>export_dta(df, filename)</code>	Export Stata <code>.dta</code> avec labels de variables.
<code>run(df_final)</code>	Appelle les deux exports, retourne les chemins.

TABLE 7.1 – Fonctions de `exporter.py`

Note

Stata impose une limite de **32 caractères** sur les noms de variables. Exemple : `satisfaction_emplacements_corbeilles` → `satisfaction_emplacements_cor`

Cette contrainte de 32 caractères est propre au format Stata et n'existe pas pour le format CSV : le fichier `.csv` conserve donc les noms de variables complets et explicites, tandis que le fichier `.dta` utilise une version tronquée mais systématiquement documentée dans le rapport QAQC, afin qu'aucune information ne soit perdue de façon silencieuse.

II qaqc.py : rapport Excel 6 onglets

Le rapport QAQC répond à un objectif de transparence : il permet à toute personne de l'UCG, même sans compétence en programmation, d'auditer la qualité du traitement effectué sur les données brutes, et de comprendre précisément quelles corrections ont été appliquées et pourquoi.

Onglet	Contenu
01_Résumé	Dimensions brutes vs traitées, taux de complétion, variables absentes
02_Anomalies_QC	Journal des 49 anomalies : type, description, N obs., action
03_Manquants	Taux par variable, trié décroissant, code couleur
04_Estimations_Primaires	Distributions de toutes les variables clés par bloc
05_Montant_Evacuation	Stats du proxy montant mensuel (moyenne, médiane, min, max)
06_Variables_Absentes	Documentation des 9 variables non collectées

TABLE 7.2 – Structure du rapport QAQC

Chaque onglet correspond à un usage distinct : l'onglet `02_Anomalies_QC` reprend directement le contenu de `qc_log` constitué par `loader.py`, tandis que l'onglet `06_Variables_Absentes` formalise, sous forme de tableau, les mêmes informations que celles présentées au chapitre 10 de ce document.

Chapitre 8

Orchestrateur run_pipeline.py

L'orchestrateur est le point d'entrée unique du pipeline en ligne de commande : c'est lui qui appelle successivement les quatre modules présentés dans les chapitres précédents, dans le bon ordre, tout en assurant une traçabilité complète de l'exécution grâce à un système de journalisation double (console et fichier horodaté).

```
1  #!/usr/bin/env python3
2  import os, sys, logging, time
3  from datetime import datetime
4
5  sys.path.insert(0, os.path.dirname(__file__))
6  import loader, builder, qaqc, exporter
7  from config import OUTPUT_DIR
8
9  os.makedirs(OUTPUT_DIR, exist_ok=True)
10 log_file = os.path.join(OUTPUT_DIR,
11     f"pipeline_{datetime.now().strftime('%Y%m%d_%H%M%S')}.log")
12 logging.basicConfig(level=logging.INFO,
13     format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
14     handlers=[logging.StreamHandler(sys.stdout),
15         logging.FileHandler(log_file, encoding="utf-8")])
16 logger = logging.getLogger("pipeline")
17
18 def run():
19     start = time.time()
20     logger.info("PIPELINE DECHETS MENAGERS -- DEMARRAGE")
21     qc_log = []
22     logger.info("[ETAPE 1/4] Chargement et controle qualite...")
23     df_raw, meta = loader.run(qc_log)
24     logger.info("[ETAPE 2/4] Construction des variables...")
25     df_final = builder.run(df_raw)
26     logger.info("[ETAPE 3/4] Export CSV et Stata...")
27     exporter.run(df_final)
28     logger.info("[ETAPE 4/4] Rapport QAQC...")
29     qaqc_path = os.path.join(OUTPUT_DIR, "QAQC_rapport.xlsx")
30     qaqc.run(df_raw, df_final, qc_log, qaqc_path)
31     elapsed = round(time.time() - start, 1)
32     logger.info(f"TERMINE -- {elapsed}s | "
```

```
33         f"{df_final.shape[0]} menages | "  
34         f"{df_final.shape[1]} variables | "  
35         f"{len(qc_log)} anomalies")  
36     return df_final, qc_log  
37  
38 if __name__ == "__main__":  
39     run()
```

Listing 8.1 – run_pipeline.py

Deux choix de conception méritent d'être soulignés. D'une part, le fichier de log est horodaté (`pipeline_YYYYMMDD_HHMMSS.log`) afin de conserver un historique complet de chaque exécution passée, sans qu'une exécution n'écrase le journal de la précédente. D'autre part, la fonction `run()` retourne à la fois la table finale et le journal d'anomalies, ce qui permet de réutiliser le pipeline de façon interactive (par exemple dans un notebook Jupyter) sans devoir relire les fichiers depuis le disque.

Chapitre 9

API FastAPI, main.py

Si l'exécution en ligne de commande convient parfaitement à un utilisateur familier de Python, elle reste un frein pour le reste de l'équipe de l'UCG. L'API FastAPI a été développée précisément pour lever cet obstacle : elle expose le pipeline complet derrière une interface web accessible depuis un simple navigateur, sans installation préalable.

I Routes

Le tableau ci-dessous liste les quatre routes exposées par l'API, chacune répondant à un besoin distinct : consulter l'interface, soumettre un fichier à traiter, obtenir des métadonnées sur le pipeline, ou consulter la documentation technique générée automatiquement par FastAPI.

Méthode	Route	Description
GET	/	Interface web HTML
POST	/api/process	Reçoit .dta, lance pipeline, retourne ZIP
GET	/api/info	Informations JSON sur le pipeline
GET	/docs	Documentation Swagger

TABLE 9.1 – Routes de l'API

La route `/docs`, générée automatiquement par FastAPI, présente un intérêt particulier pour un développeur souhaitant intégrer le pipeline dans un autre système : elle offre une interface interactive pour tester chaque route sans écrire de code.

II Flux POST /api/process

La route `/api/process` concentre toute la logique d'intégration du pipeline dans l'API. Chaque requête est traitée de façon totalement isolée des autres, ce qui permet en principe à plusieurs utilisateurs de soumettre un fichier simultanément sans risque d'interférence entre leurs traitements respectifs.

1. Validation : vérifier l'extension `.dta`
2. Création d'un répertoire temporaire isolé
3. Sauvegarde du fichier uploadé
4. Patch dynamique de `config.INPUT_DIR` et `config.OUTPUT_DIR`
5. Exécution : loader → builder → export → qaqc
6. Compression dans un ZIP en mémoire (BytesIO)
7. Retour via `StreamingResponse`

Le répertoire temporaire créé à l'étape 2 est systématiquement supprimé une fois la réponse envoyée, quel que soit le résultat du traitement (succès ou échec) : cette hygiène de nettoyage évite l'accumulation de fichiers résiduels sur le serveur au fil des requêtes.

III Code source

Le code ci-dessous met en œuvre le flux décrit ci-dessus. Un point technique mérite attention : les modules `loader` et `exporter` sont rechargés dynamiquement (`importlib.reload`) à chaque requête, afin de tenir compte des chemins `INPUT_DIR` et `OUTPUT_DIR` propres à cette requête précise.

```

1 import io, os, sys, tempfile, zipfile, logging
2 from pathlib import Path
3 from fastapi import FastAPI, File, UploadFile, HTTPException
4 from fastapi.responses import HTMLResponse, StreamingResponse,
   JSONResponse
5 from fastapi.templating import Jinja2Templates
6 from fastapi.requests import Request
7
8 sys.path.insert(0, os.path.join(os.path.dirname(__file__), "src"))
9 app = FastAPI(title="Pipeline Dechets Menagers", version="1.0.0")
10 templates = Jinja2Templates(directory="templates")
11
12 @app.get("/", response_class=HTMLResponse)
13 async def index(request: Request):
14     return templates.TemplateResponse("index.html", {"request":
15         request})
16
17 @app.post("/api/process")
18 async def process_file(file: UploadFile = File(...)):
19     if not file.filename.endswith(".dta"):
20         raise HTTPException(400, "Format invalide : .dta requis.")
21     tmp = Path(tempfile.mkdtemp())

```

```

21 (tmp/"input").mkdir(); (tmp/"output").mkdir()
22 (tmp/"input"/"Base.dta").write_bytes(await file.read())
23 try:
24     import importlib, config, loader, builder, qaqc, exporter
25     config.INPUT_DIR = str(tmp/"input")
26     config.OUTPUT_DIR = str(tmp/"output")
27     config.INPUT_FILE = "Base.dta"
28     importlib.reload(loader); importlib.reload(exporter)
29     qc_log = []
30     df_raw, _ = loader.run(qc_log)
31     df_final = builder.run(df_raw)
32     csv = str(tmp/"output"/"dataset_dechets.csv")
33     dta = str(tmp/"output"/"dataset_dechets.dta")
34     xlsx = str(tmp/"output"/"QAQC_rapport.xlsx")
35     df_final.to_csv(csv, index=False, encoding="utf-8-sig")
36     import pyreadstat
37     df_s, _ = exporter._prepare_for_stata(df_final)
38     pyreadstat.write_dta(df_s, dta)
39     qaqc.run(df_raw, df_final, qc_log, xlsx)
40 except Exception as e:
41     raise HTTPException(500, str(e))
42 buf = io.BytesIO()
43 with zipfile.ZipFile(buf, "w", zipfile.ZIP_DEFLATED) as zf:
44     zf.write(csv, "dataset_dechets.csv")
45     zf.write(dta, "dataset_dechets.dta")
46     zf.write(xlsx, "QAQC_rapport.xlsx")
47 buf.seek(0)
48 return StreamingResponse(buf, media_type="application/zip",
49     headers={"Content-Disposition":
50         "attachment; filename=resultats_pipeline.zip"})
51
52 @app.get("/api/info")
53 async def info():
54     return JSONResponse({"pipeline": "ENSAE/UCG 2018", "version": "1.0.0",
55         "format": ".dta", "variables": 132, "menages": 2474})
56
57 if __name__ == "__main__":
58     import uvicorn
59     uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

Listing 9.1 – main.py

Cette approche par rechargement de modules, bien que fonctionnelle pour un usage à faible concurrence comme celui de l'UCG, présente une limite connue : elle n'est pas strictement sûre en environnement fortement concurrent, où plusieurs requêtes simul-

tanées pourraient théoriquement interférer sur les variables globales du module `config`. Cette limite est documentée ici pour orienter une évolution future de l'architecture si le volume d'utilisation venait à augmenter significativement.

Troisième partie

VARIABLES, UTILISATION ET DÉ- PLOIEMENT

Chapitre 10

Variables produites et absentes

I Récapitulatif

Le tableau ci-dessous reprend la répartition des 132 variables analytiques entre les quatre blocs détaillés au chapitre 6. Cette vision synthétique permet de mesurer d'un coup d'œil le poids relatif de chaque thématique dans la table finale : les déchets ménagers concentrent, sans surprise, près des deux tiers des variables produites.

Bloc	Nb	Thème
A (Ménage)	12	Géographie, habitat, proxy niveau de vie
B (Chef de ménage)	19	Démographie, instruction, alphabétisation
C (Déchets)	87	Sources, stockage, évacuation, infrastructure
D (Conséquences)	14	Maladies déclarées, indicateurs sanitaires
Total	132	

TABLE 10.1 – Variables analytiques produites

II Variables absentes

Documenter explicitement ce qui n'est pas mesuré est tout aussi important que de documenter ce qui l'est : cela évite qu'un futur utilisateur de la table ne suppose à tort qu'une variable manquante correspond à une erreur de traitement, alors qu'il s'agit en réalité d'une limite du questionnaire source.

Variable	Statut	Note
<code>region</code>	ABSENT	Zone entièrement urbaine (Dakar).
<code>departement</code>	ABSENT	Non extrait comme variable séparée.
<code>milieu_residence</code>	ABSENT	Toute la zone est urbaine.
<code>depenses_menage</code>	PROXY	Score proxy via I_6 + I_7 + II_24.
<code>actifs_menage</code>	PROXY	Inclus dans proxy_niveau_vie (0–3).
<code>revenu_CM</code>	PROXY	Proxy via statut et paiement déchets.
<code>situation_matr._CM</code>	ABSENT	Non collectée dans ce questionnaire.
<code>statut_emploi_CM</code>	ABSENT	Non collecté.
<code>secteur_emploi_CM</code>	ABSENT	Non collecté.

TABLE 10.2 – Variables absentes et documentation

Les variables marquées PROXY doivent être manipulées avec précaution dans les analyses ultérieures : elles offrent une indication utile, mais ne remplacent pas une mesure directe et ne devraient pas être présentées comme telle dans un rapport destiné à un public non averti.

Chapitre 11

Guide d'utilisation et déploiement

Ce dernier chapitre s'adresse directement à la personne qui souhaite faire fonctionner le pipeline, que ce soit pour le tester en local, l'exécuter en ligne de commande, ou le déployer en ligne afin de le rendre accessible à toute l'équipe de l'UCG.

I Installation

La première étape, commune à tous les modes d'utilisation, consiste à récupérer le projet et à installer ses dépendances Python.

```
1 # Dézipper le projet
2 cd Groupe_Insalubrite_traitement_donnees
3
4 # Installer les dépendances
5 pip install -r requirements.txt
6
7 # Copier Base.dta dans input/
```

Listing 11.1 – Installation

II Option A : ligne de commande

Ce mode d'exécution convient particulièrement à un utilisateur familier de Python souhaitant automatiser le traitement, par exemple dans un script planifié ou une tâche récurrente.

```
1 cd src
2 python run_pipeline.py
3 # Resultats dans output/
```

Listing 11.2 – Pipeline en ligne de commande

III Option B : plateforme web locale

Ce second mode convient davantage à un utilisateur occasionnel : il suffit de lancer un serveur local puis d'utiliser l'interface web dans un navigateur, sans avoir à taper la moindre ligne de commande Python.

```
1 # Depuis la racine du projet
2 python -m uvicorn main:app --reload
3 # Ouvrir http://127.0.0.1:8000
```

Listing 11.3 – Plateforme web

IV Déploiement sur Render

Pour rendre le pipeline accessible à l'ensemble de l'équipe de l'UCG sans que chacun n'ait à installer Python sur sa propre machine, le projet peut être déployé sur une plateforme d'hébergement en ligne telle que Render, qui lit automatiquement le fichier de configuration [render.yaml](#) inclus dans le dépôt.

```
1 git init && git add .
2 git commit -m "pipeline dechets v1.0"
3 git push origin main
4 # Sur render.com : New > Web Service > connecter le repo
5 # Render lit render.yaml automatiquement
```

Listing 11.4 – Déploiement

```
services:
  - type: web
    name: pipeline-dechets-menagers
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
    envVars:
      - key: PYTHON_VERSION
        value: 3.11.0
```

Listing 11.5 – render.yaml

V Dépendances Python

Le tableau ci-dessous liste l'ensemble des paquets Python nécessaires au fonctionnement du pipeline, avec la version testée et validée pour chacun d'entre eux, ainsi que

leur rôle respectif dans la chaîne de traitement.

Package	Version	Usage
fastapi	0.115.0	Framework API REST
uvicorn[standard]	0.30.6	Serveur ASGI
python-multipart	0.0.9	Gestion uploads
jinja2	3.1.4	Templates HTML
pyreadstat	1.3.1	Lecture/écriture Stata
pandas	2.2.2	Manipulation DataFrames
numpy	1.26.4	Calculs numériques
openpyxl	3.1.2	Génération Excel

TABLE 11.1 – requirements.txt

1. Placer le nouveau `.dta` dans `input/`
2. Vérifier les codes dans `src/config.py`
3. Relancer : `python src/run_pipeline.py`
4. Aucune modification des autres modules n'est nécessaire

En résumé, ce pipeline a été conçu pour rester simple à faire évoluer : toute la connaissance spécifique au questionnaire réside dans `config.py`, tandis que la mécanique de traitement, éprouvée sur les 2 474 ménages de l'édition 2018, reste stable d'une édition à l'autre de l'enquête.